

OTT Platform Analytics – StreamFlix

Task 1: Identify users who watched more than the platform's average watch hours.

```
41
42  -- Task 1: Identify users who watched more than the platform's average watch hours.
43
44  WITH cte AS (
45      SELECT user_name, ROUND(AVG(watch_hours) over(),2) AS avg_watch_hours, watch_hours
46      FROM ott_viewership
47  )
48  SELECT * FROM cte
49  WHERE watch_hours > avg_watch_hours;
50
```

Data Output Messages Notifications

	user_name character varying (50)	avg_watch_hours numeric	watch_hours integer
1	Priya	21.25	25
2	Riya	21.25	30
3	Karan	21.25	22
4	Sohan	21.25	28
5	Arjun	21.25	24
6	Rohit	21.25	26

Task 2: Determine whether any user from the West region watched “The Witcher”.

assessment.sql* x public.ott_viewers... x

Learning_PG/postgres@PostgreSQL 18

Query Query History

```
51
52  -- Task 2: Determine whether any user from the West region watched “The Witcher”
53  WITH south_region AS(
54      SELECT *
55      FROM ott_viewership
56      WHERE region = 'West'
57  )
58  SELECT * FROM south_region
59  WHERE show_name= 'The Witcher';
60
61
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

view_id [PK] integer	user_name character varying (50)	region character varying (30)	show_name character varying (50)	genre character varying (30)	watch_hours integer	subscription_amount integer	watch_month character varying (15)
1	9 Pooja	West	The Witcher	Fantasy	16	499	May

Task 3: Identify regions where at least one user watched more than 25 hours.

Query Query History

```
60
61
62 -- Task 3: Identify regions where at least one user watched more than 25 hours.
63 WITH watchhour AS (
64     SELECT *
65     FROM ott_viewership
66     WHERE watch_hours>25
67 )
68 SELECT user_name,REGION,watch_hours
69 FROM watchhour;
70
```

Data Output Messages Notifications

	user_name character varying (50)	region character varying (30)	watch_hours integer
1	Riya	East	30
2	Sohan	South	28
3	Rohit	West	26

Task 4: Create a reusable query output/result table showing:

- Region
- Total Watch Hours
- Total Subscription Revenue Store this query output/result table for future use.

```
70
71 -- Task 4:
72 -- Create a reusable query output/result table showing:
73 -- Region
74 -- Total Watch Hours
75 -- Total Subscription Revenue
76 -- Store this query output/result table for future use.
77
78 CREATE OR REPLACE VIEW region_summary AS (
79     SELECT region,
80            SUM(watch_hours) AS total_watch_hours,
81            SUM(subscription_amount) AS total_subs_revenue
82     FROM ott_viewership
83     GROUP BY region
84 );
85 SELECT * FROM region_summary;
86
87
88
```

Data Output Messages Notifications

	region character varying (30)	total_watch_hours bigint	total_subs_revenue bigint
1	East	67	2397
2	South	60	1997
3	West	67	2197
4	North	61	1897

Showing rows: 1 to 4 Page

Task 5: Rank all shows based on total watch hours.

```
97
98 -- Task 5: Rank all shows based on total watch hours
99
100 SELECT show_name,
101         SUM(watch_hours) AS total_watch_hours,
102         DENSE_RANK() OVER( ORDER BY SUM(watch_hours) DESC )
103 FROM ott_viewership
104 GROUP BY show_name;
105
106
107
```

Data Output Messages Notifications

	show_name character varying (50)	total_watch_hours bigint	dense_rank bigint
1	Stranger Things	87	1
2	Wednesday	66	2
3	Money Heist	56	3
4	The Witcher	46	4

Task 6: Assign a unique position to each user based on watch hours.

```
97 -- Task 6: Assign a unique position to each user based on watch hours
98 WITH top_viewers AS (
99     SELECT user_name, SUM(watch_hours) AS total_watch_hours
100 FROM ott_viewership
101 GROUP BY user_name
102 ORDER BY total_watch_hours DESC
103 )
104 SELECT *,
105        RANK() OVER (ORDER BY total_watch_hours DESC)
106 FROM top_viewers;
107
108
```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1

	user_name character varying (50)	total_watch_hours bigint	rank bigint
1	Riya	30	1
2	Sohan	28	2
3	Rohit	26	3
4	Priya	25	4
5	Arjun	24	5
6	Karan	22	6
7	Neha	20	7
8	Meera	19	8
9	Vivek	18	9
10	Pooja	16	10
11	Rahul	15	11
12	Amit	12	12

Task 7: Rank all regions based on subscription revenue. If two regions have the same revenue, they should share the same rank.

```

109
110 -- Task 7: Rank all regions based on subscription revenue.
111 -- If two regions have the same revenue, they should share the same rank.
112
113 SELECT *,
114         DENSE_RANK() OVER(ORDER BY total_subs_revenue DESC ) AS rnk
115 FROM region_summary; --Utilizing region_summary view we created before
116
117
118
119
120

```

Data Output Messages Notifications

	region character varying (30)	total_watch_hours bigint	total_subs_revenue bigint	rnk bigint
1	East	67	2397	1
2	West	67	2197	2
3	South	60	1997	3
4	North	61	1897	4

Task 8: For every month, display the previous month's subscription revenue.

```

120
121 -- Task 8: For every month, display the previous month's subscription revenue.
122 CREATE OR REPLACE VIEW months_summary AS (
123 SELECT CASE WHEN watch_month = 'Jan' THEN 1
124             WHEN watch_month = 'Feb' THEN 2
125             WHEN watch_month = 'Mar' THEN 3
126             WHEN watch_month = 'Apr' THEN 4
127             WHEN watch_month = 'May' THEN 5
128             WHEN watch_month = 'Jun' THEN 6
129             END AS month_no,
130            watch_month,
131            SUM(subscription_amount) AS "Subscription Revenue"
132 FROM ott_viewership
133 GROUP BY watch_month
134 ORDER BY month_no
135 );
136 SELECT *, LAG("Subscription Revenue") OVER() AS previous_month_revenue
137 FROM months_summary;
138

```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1

	month_no integer	watch_month character varying (15)	Subscription Revenue bigint	previous_month_revenue bigint
1	1	Jan	998	[null]
2	2	Feb	1398	998
3	3	Mar	1498	1398
4	4	Apr	1698	1498
5	5	May	1198	1698
6	6	Jun	1698	1198

Task 9: For every month, display the next month's subscription revenue.

```
139
140 -- Task 9: For every month, display the next month's subscription revenue.
141 SELECT *,
142       LEAD("Subscription Revenue") OVER() AS next_month_revenue
143 FROM months_summary; -- utilizing months_summary VIEW
144
145
```

	month_no integer	watch_month character varying (15)	Subscription Revenue bigint	next_month_revenue bigint
1	1	Jan	998	1398
2	2	Feb	1398	1498
3	3	Mar	1498	1698
4	4	Apr	1698	1198
5	5	May	1198	1698
6	6	Jun	1698	[null]

Task 10: Create a query output/result table that shows:

- Show Name
- Total Watch Hours
- Show Rank Then Display Top 3 Shows on Platform

```
121
122 -- Task 10:
123 -- Create a query output/result table that shows:
124 -- Show Name
125 -- Total Watch Hours
126 -- Show Rank
127 -- Then display only the Top 3 shows on the platform
128 CREATE OR REPLACE VIEW show_summary AS (
129     SELECT show_name,
130           SUM(watch_hours) AS total_watch_hours,
131           DENSE_RANK() OVER (ORDER BY SUM(watch_hours) DESC)
132     FROM ott_viewership
133     GROUP BY show_name
134 );
135
136 SELECT * FROM show_summary
137 LIMIT 3;
138
```

	show_name character varying (50)	total_watch_hours bigint	dense_rank bigint
1	Stranger Things	87	1
2	Wednesday	66	2
3	Money Heist	56	3

Task 11: Create a query output/result table that shows:

- Region
- Total Revenue
- Revenue Rank

```
163
164 -- Task 11:
165 -- Create a query output/result table that shows:
166 -- Region
167 -- Total Revenue
168 -- Revenue Rank
169 -- Display only the Top 2 performing regions.
170
171 SELECT region, total_subs_revenue AS total_revenue ,
172         DENSE_RANK() OVER(ORDER BY total_subs_revenue DESC ) AS Revenue_Rank
173 FROM region_summary --Utilizing region_summary view we created before
174 LIMIT 2;
175
176
```

Data Output Messages Notifications

	region character varying (30)	total_revenue bigint	revenue_rank bigint
1	East	2397	1
2	West	2197	2

Task 12: Identify the most watched show in each region.

```
151
152
153 -- Task 12: Identify the most watched show in each region
154
155 WITH show_region AS (
156     SELECT region, show_name, SUM(watch_hours),
157           DENSE_RANK() OVER(PARTITION BY REGION ORDER BY SUM(watch_hours) DESC ) AS rnk
158     FROM ott_viewership
159     GROUP BY region, show_name)
160
161
162 SELECT *
163 FROM show_region
164 WHERE rnk <2;
165
```

Data Output Messages Notifications

	region character varying (30)	show_name character varying (50)	sum bigint	rnk bigint
1	East	The Witcher	30	1
2	North	Money Heist	24	1
3	South	Money Heist	32	1
4	West	Stranger Things	26	1

Task 13: Write any three business insights from the dataset.

- **April** is Best Performing Month w.r.t **Watch Hours & Subscription Revenue**.
- **“Stranger Things”** is Most Watched Show on Platform.
- **East** Region leads in **Watch Hours & Subscription Revenue**.